

Assignment 2

Linearly Solvable Markov Decision Processes

Overview

Consider a finite-state MDP with deterministic transitions $x' = F(x, u)$, running cost $C(x, u)$, and a reference policy $R(u|x)$. The KL-regularized optimal policy minimizes

$$J(Q) = \sum_{k=0}^{T-1} \mathbb{E}_Q \left[C(x_k, u_k) + \alpha \log \frac{Q(u_k|x_k)}{R(u_k|x_k)} \right] + \mathbb{E}_Q [C_{\text{exit}}(x_T)].$$

Under deterministic transitions, this problem is *linearly solvable*: define the desirability $Z_k(x) = \exp(-V_k(x)/\alpha)$ and the transition matrix

$$M(x, x') = \sum_u \mathbf{1}_{x'=F(x,u)} R(u|x) \exp(-C(x, u)/\alpha). \quad (7.27)$$

Then $Z_k = M Z_{k+1}$ (Algorithm 12), and the optimal policy is

$$Q^*(u|x) = \frac{R(u|x) \exp(-C(x, u)/\alpha) Z_{k+1}(F(x, u))}{Z_k(x)}. \quad (7.29)$$

Alternatively, $Z_k(x)$ can be estimated by forward Monte Carlo sampling under R without constructing M at all (Algorithm 13).

In this assignment you will implement both approaches on a gridworld and compare them.

Part A: Backward Recursion

Code is in `lmdp/backward.py`.

1. Desirability matrix (Eq. 7.27)

Implement `build_M_matrix(grid, alpha)`.

Construct M by looping over all states x and actions u . Use `grid.reference_policy(x)` for $R(u|x)$, `grid.step(x, u)` for $F(x, u)$, and `grid.cost(x, u)` for $C(x, u)$.

2. Backward pass (Algorithm 12)

Implement `backward_recursion(Z_T, M, T)`.

Starting from terminal desirability $Z_T(x) = \exp(-C_{\text{exit}}(x)/\alpha)$, compute $Z_k = M Z_{k+1}$ for $k = T-1, \dots, 0$.

3. Policy reconstruction (Eq. 7.29)

Implement `reconstruct_policy(grid, Z, alpha)`.

Recover $Q^*(u_k|x_k)$ from the desirability vectors. Normalize so that $\sum_u Q^*(u|x) = 1$. If $Z_k(x) \approx 0$, fall back to uniform.

Part B: Forward Monte Carlo

Code is in `lmdp/forward_mc.py`.

4. Path rewards (Eq. 7.37)

Implement `compute_path_rewards(grid, states, actions, alpha)`.

For each sample path i , accumulate the running cost plus terminal cost, then compute $r(i) = \exp(-C_{k:T}(\text{path}_i)/\alpha)$. Subtract $C_{\min}$ before exponentiating for numerical stability.

5. Per-action reward (Eq. 7.40)

Implement `empirical_reward_per_action(rewards, first_actions, n_actions)`.

For each action u , average the rewards of paths whose first action equals u :

$$r(x_k, u) = \frac{1}{N} \sum_{i \in I(u)} r(i), \quad I(u) = \{i : u_k^{(i)} = u\}.$$

6. Approximate policy (Eq. 7.43)

Implement `approximate_policy(r_per_action)`.

Normalize $r(x, u)$ to obtain $Q^*(u|x) = r(x, u) / \sum_{u'} r(x, u')$. If the denominator is near zero, fall back to uniform.

Experiments

All gridworld examples run on a discretized 2D environment with Gaussian obstacle costs, a unit step cost $C(x, u) = 1$ per move, zero terminal cost at the goal, and high terminal cost elsewhere. The default environment is `three_mountains` (three Gaussian peaks between start and goal). Use `--env config/environments/<name>.yaml` to try other environments (`landing_site`, `simple_goal`).

Backward recursion

Computes the exact desirability $Z_k(x)$ via the linear recursion $Z_k = MZ_{k+1}$ and reconstructs the optimal policy:

```
python examples/discrete/gridworld_backward.py
python examples/discrete/gridworld_backward.py --alpha 1.0
python examples/discrete/gridworld_backward.py --alpha 100.0
```

Forward Monte Carlo

Estimates $Z_k(x)$ by sampling N paths under the reference policy R without constructing M :

```
python examples/discrete/gridworld_forward_mc.py
python examples/discrete/gridworld_forward_mc.py --N 50
python examples/discrete/gridworld_forward_mc.py --N 5000
```

Determinism breaking

Runs on a 5×5 gridworld with two obstacles. Compares the linear recursion (exact under deterministic transitions) against the true nonlinear Bellman solution and forward MC under stochastic transitions. With slip probability p , the agent executes a uniformly random action with probability p instead of the intended one, making $P(x'|x, u)$ stochastic:

```
python examples/discrete/determinism_breaking.py --slip 0.0
python examples/discrete/determinism_breaking.py --slip 0.2
```

1. Run the backward solver with $\alpha \in \{1, 10, 100\}$. How does α affect the sharpness of the policy? Explain in terms of the KL penalty.
2. Run the forward MC solver with $N \in \{50, 500, 5000\}$ samples per step. Compare the resulting trajectory cost to the exact backward solution. How does the policy quality scale with N ?
3. Run `determinism_breaking.py` with `--slip 0.0` and `--slip 0.2`. Why does the linear recursion $Z_k = MZ_{k+1}$ overestimate the desirability under stochastic transitions? Which inequality is responsible?

Deliverables

- Completed `lmdp/backward.py` and `lmdp/forward_mc.py`.
- Plots from the backward, forward MC, and determinism breaking examples.
- Written responses to experiment questions 1–3.